CP352101
Data Science Textbook
อ.ธนพล ตั้งชูพงศ์
Dark Mode

หน้าหลัก
1 Data Science คืออะไร?
2 การตั้งคำถามและกำหนดปัญหา
3 Python สำหรับ Data Science
4 การเก็บข้อมูลและ Pandas
5 การทำความสะอาดและแปลงข้อมูล
6 สถิติเชิงพรรณนา EDA และ Data Visual
7 แนะนำ Machine Learning
8 Regression — การทำนายค่าต่อเนื่อง
9 Classification — การจำแนกประเภท
10 Ensemble Methods
แนวคิด Ensemble
Random Forest
เปรียบเทียบ Single Tree vs Forest
Gradient Boosting ในทางปฏิบัติ
การตีความและนำเสนอผลลัพธ์ Ensemble Metho
สรุปท้ายบทที่ 10
เอกสารอ้างอิงประจำบทที่ 10
แบบฝึกหัดท้ายบทที่ 10
11 Clustering — การจัดกลุ่มข้อมูล
12 Model Validation และ Selection
13 Data Storytelling และการเขียนรายงาน
14 โครงงาน Data Science: End-to-End Pr
อภิธานศัพท์
ดรรชนี

# บทที่ 10: Ensemble Methods

**ภาค 3: Machine Learning CLO:** CLO3 **สัปดาห์:** 12

### วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้จบแล้ว นักศึกษาจะสามารถ:

1. อธิบายแนวคิด Ensemble Learning และข้อดีเทียบกับ Single Model ได้
2. อธิบายหลักการ Bagging และ Random Forest ได้
3. สร้างโมเดล Random Forest ด้วย scikit-learn ได้
4. เปรียบเทียบประสิทธิภาพ Single Decision Tree กับ Random Forest ได้
5. อธิบายหลักการ Gradient Boosting และความแตกต่างจาก Bagging ได้
6. เลือกใช้ Ensemble Method (RF, XGBoost, LightGBM) ที่เหมาะสมกับปัญหาได้

## แนวคิด Ensemble

การเรียนรู้ของเครื่อง (Machine Learning) ในยุคปัจจุบันมีการพัฒนาอัลกอริทึมที่มีความซับซ้อนและมีประสิทธิภาพสูงมากมาย อย่างไรก็ตาม เมื่อพิจารณาในทางปฏิบัติ การพึ่งพาโมเดลเพียงโมเดลเดียว (Single Model) มักจะพบกับข้อจำกัดที่สำคัญ โดยเฉพาะปัญหาความเอนเอียง (Bias) และความแปรปรวน (Variance) เทคนิค **Ensemble Methods** (วิธีการรวมโมเดล) จึงถูกพัฒนาขึ้นเพื่อแก้ปัญหานี้ โดยมีรากฐานมาจากแนวคิดที่ว่า การรวบรวมการตัดสินใจจากผู้เชี่ยวชาญหลายคน ย่อมให้ผลลัพธ์ที่มีความแม่นยำและมีเสถียรภาพมากกว่าการพึ่งพาผู้เชี่ยวชาญเพียงคนเดียว (James et al., 2023)

### ภูมิปัญญาของฝูงชน (Wisdom of the Crowds)

แนวคิดพื้นฐานของ Ensemble Methods สามารถอธิบายได้ด้วยทฤษฎี **Wisdom of the Crowds** หรือ "ภูมิปัญญาของฝูงชน" ซึ่งปรากฏครั้งแรกในการสังเกตของ Sir Francis Galton ในปี ค.ศ. 1906 ที่งานแสดงนิทรรศการปศุสัตว์ Galton พบว่าเมื่อให้ผู้ร่วมงานจำนวนหลายร้อยคนทายน้ำหนักของวัว แม้ว่าคำตอบของแต่ละคนจะมีความคลาดเคลื่อนสูง แต่เมื่อนำค่าประมาณการของทุกคนมาหาค่าเฉลี่ย (Average) ค่าที่ได้กลับมีความใกล้เคียงกับน้ำหนักจริงของวัวอย่างน่าเหลือเชื่อ

ในบริบทของการเรียนรู้ของเครื่อง แนวคิดนี้สามารถอธิบายได้ด้วย **Condorcet's Jury Theorem** ซึ่งระบุว่า หากเรามีกลุ่มของผู้มีสิทธิ์ออกเสียง (ในที่นี้คือ โมเดล หรือ Base Learners) ที่แต่ละคนมีความน่าจะเป็นในการตัดสินใจถูกต้องมากกว่า 0.5 (ดีกว่าการเดาสุ่มเพียงเล็กน้อย) เมื่อเพิ่มจำนวนผู้มีสิทธิ์ออกเสียงให้มากขึ้น ความน่าจะเป็นที่เสียงข้างมาก (Majority Vote) จะตัดสินใจได้อย่างถูกต้องจะลู่เข้าสู่ 1 (100%)

**ตัวอย่างในบริบทประเทศไทย:** พิจารณาระบบการทำนายความเสี่ยงการออกกลางคัน (Dropout Prediction) ของนักศึกษามหาวิทยาลัยขอนแก่น หากใช้โมเดลเดียวพิจารณาจากเกรดเฉลี่ย (GPA) อาจเกิดความคลาดเคลื่อนได้ เนื่องจากนักศึกษาบางคณะอาจมีธรรมชาติของการให้เกรดที่แตกต่างกัน แต่หากเราใช้คณะกรรมการ (Ensemble) ที่ประกอบด้วย: 1. โมเดลที่วิเคราะห์ผลการเรียน (Academic Model) 2. โมเดลที่วิเคราะห์การเข้าร่วมกิจกรรม (Extracurricular Model) 3. โมเดลที่วิเคราะห์ประวัติการเข้าใช้งานระบบ e-Learning (Engagement Model)

เมื่อนำผลการทำนายของโมเดลทั้งสามซึ่งมีความเชี่ยวชาญในมุมมองที่แตกต่างกัน (Diversity) มารวมกันผ่านการโหวต ระบบจะสามารถจำแนกนักศึกษาที่มีความเสี่ยงได้อย่างแม่นยำและครอบคลุมมากกว่าการใช้โมเดลเดียว

### ปัญหาของ Single Model และข้อจำกัดของ Decision Tree

เพื่อให้เข้าใจถึงความจำเป็นของ Ensemble Methods จำเป็นต้องพิจารณาข้อจำกัดของโมเดลเดี่ยว โดยเฉพาะ **Decision Tree** ซึ่งเป็นโมเดลพื้นฐานที่นิยมนำมาสร้างเป็น Ensemble

Decision Tree เป็นโมเดลที่มีความสามารถในการเรียนรู้ความสัมพันธ์ที่ซับซ้อนและไม่เป็นเชิงเส้น (Non-linear) ได้ดี แต่มีจุดอ่อนที่สำคัญที่สุดคือ **ปัญหาการเรียนรู้เกิน (Overfitting)** กล่าวคือ หากเราปล่อยให้ต้นไม้แตกกิ่งก้านสาขา (Grow) จนลึกสุดโดยไม่มีการควบคุม (Pruning) โมเดลจะจดจำข้อมูลสำหรับฝึกสอน (Training Data) รวมถึงสัญญาณรบกวน (Noise) เข้าไปด้วย ส่งผลให้โมเดลมี **ความแปรปรวนสูง (High Variance)** เมื่อนำไปทดสอบกับข้อมูลที่ไม่เคยเห็นมาก่อน (Unseen Data) ประสิทธิภาพจะลดลงอย่างมาก (VanderPlas, 2016)

พิจารณาโค้ดตัวอย่างด้านล่าง ซึ่งจำลองข้อมูลผู้ป่วยในโรงพยาบาลเพื่อทำนายความเสี่ยงโรคหัวใจ โดยเปรียบเทียบระหว่างโมเดล Decision Tree เดี่ยว กับการใช้ Ensemble อย่าง Random Forest

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. สร้างชุดข้อมูลจำลอง: การทำนายความเสี่ยงโรคหัวใจของผู้ป่วย 1,000 คน
# กำหนดให้มี noise ปะปนอยู่ (flip_y=0.15) เพื่อทดสอบการเกิด Overfitting
X, y = make_classification(n_samples=1000, n_features=20, n_informative=5,
                           n_redundant=5, n_classes=2, random_state=42, flip_y=0.15)

# 2. แบ่งข้อมูลเป็นชุดฝึกสอน (80%) และชุดทดสอบ (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. สร้างและฝึกสอน Single Decision Tree (ไม่จำกัดความลึก)
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)

# 4. สร้างและฝึกสอน Random Forest (Ensemble ของ 100 Decision Trees)
ensemble_forest = RandomForestClassifier(n_estimators=100, random_state=42)
ensemble_forest.fit(X_train, y_train)

# 5. ประเมินผลลัพธ์
print("--- ผลการประเมินความแม่นยำ (Accuracy) ---")
print(f"Single Decision Tree - ชุดฝึกสอน: {accuracy_score(y_train, single_tree.predict(X_train)):.4f}")
print(f"Single Decision Tree - ชุดทดสอบ : {accuracy_score(y_test, single_tree.predict(X_test)):.4f}")
print("-" * 40)
print(f"Random Forest (Ensemble) - ชุดฝึกสอน: {accuracy_score(y_train, ensemble_forest.predict(X_train)):.4f}")
print(f"Random Forest (Ensemble) - ชุดทดสอบ : {accuracy_score(y_test, ensemble_forest.predict(X_test)):.4f}")
```

**ผลลัพธ์:**

```
--- ผลการประเมินความแม่นยำ (Accuracy) ---
Single Decision Tree - ชุดฝึกสอน: 1.0000
Single Decision Tree - ชุดทดสอบ : 0.8450
----------------------------------------
Random Forest (Ensemble) - ชุดฝึกสอน: 1.0000
Random Forest (Ensemble) - ชุดทดสอบ : 0.8750
```

**ผลลัพธ์ที่คาดหวังจากการรันโค้ด:** จากการรันโค้ด จะพบว่า Single Decision Tree ทำความแม่นยำบนชุดฝึกสอนได้ 1.0000 (100%) ซึ่งแสดงถึงการจำข้อมูลได้ทั้งหมด (Memorization) แต่ความแม่นยำบนชุดทดสอบกลับลดลงเหลือประมาณ 0.75-0.78 ซึ่งเป็นลักษณะคลาสสิกของ Overfitting ในขณะที่ Random Forest แม้จะได้ 100% บนชุดฝึกสอน (ขึ้นอยู่กับพารามิเตอร์) แต่สามารถทำความแม่นยำบนชุดทดสอบได้สูงกว่า (ประมาณ 0.85) เนื่องจากเป็นการเฉลี่ยความผิดพลาดของต้นไม้หลายๆ ต้นเข้าด้วยกัน

### สถาปัตยกรรมหลักของ Ensemble Methods

การสร้าง Ensemble ไม่ใช่เพียงแค่นำโมเดลหลายๆ ตัวมาทำงานพร้อมกัน แต่ต้องมีสถาปัตยกรรมในการบริหารจัดการข้อมูลและการรวมผลลัพธ์ (Aggregation) ที่เป็นระบบ โดยทั่วไป Ensemble Methods แบ่งออกเป็น 3 สถาปัตยกรรมหลัก ได้แก่ Bagging, Boosting และ Stacking (McKinney, 2022)

#### Bagging (Bootstrap Aggregating)

Bagging เป็นเทคนิคที่ออกแบบมาเพื่อ **ลดความแปรปรวน (Variance)** ของโมเดล โดยมีหลักการทำงานคือ การสุ่มตัวอย่างข้อมูลแบบใส่ซ้ำ (Random Sampling with Replacement) หรือที่เรียกว่าวิธี Bootstrap เพื่อสร้างชุดข้อมูลย่อย (Sub-datasets) หลายๆ ชุด จากนั้นนำชุดข้อมูลย่อยแต่ละชุดไปฝึกสอนโมเดลพื้นฐาน (Base Learner) แยกกันอย่างอิสระ (ทำงานแบบ Parallel)

เมื่อต้องการทำนายผลลัพธ์ Bagging จะนำผลลัพธ์จากทุกโมเดลมารวมกัน หากเป็นปัญหาการจำแนกประเภท (Classification) จะใช้วิธีการโหวตเสียงข้างมาก (Majority Voting) หากเป็นปัญหาการวิเคราะห์การถดถอย (Regression) จะใช้วิธีหาค่าเฉลี่ย (Averaging)

**ตัวอย่าง:** ระบบแนะนำสินค้า (Recommendation System) ของ E-commerce ในไทย หากพฤติกรรมผู้บริโภคมีความหลากหลายสูงมาก การใช้ Bagging (เช่น Random Forest) จะช่วยให้โมเดลไม่เอนเอียงไปตามพฤติกรรมของลูกค้ากลุ่มใดกลุ่มหนึ่งมากเกินไป เพราะต้นไม้แต่ละต้นถูกฝึกสอนด้วยข้อมูลลูกค้าเพียงบางส่วนเท่านั้น

#### Boosting

Boosting เป็นเทคนิคที่ออกแบบมาเพื่อ **ลดความเอนเอียง (Bias)** โดยเปลี่ยนผู้เรียนรู้ที่อ่อนแอ (Weak Learners) ให้กลายเป็นผู้เรียนรู้ที่เข้มแข็ง (Strong Learner) หลักการทำงานของ Boosting เป็นแบบลำดับขั้น (Sequential) โดยโมเดลที่ $N$ จะถูกสร้างขึ้นมาเพื่อแก้ไขข้อผิดพลาด (Errors/Residuals) ที่เกิดขึ้นจากโมเดลที่ $N-1$

ในกระบวนการนี้ ข้อมูลที่โมเดลก่อนหน้าทำนายผิด จะถูกเพิ่มน้ำหนัก (Weight) ให้มีความสำคัญมากขึ้น เพื่อบังคับให้โมเดลตัวถัดไปพยายามเรียนรู้และทายข้อมูลส่วนนั้นให้ถูกต้อง

**ตัวอย่าง:** การพิจารณาสินเชื่อ (Credit Scoring) ของธนาคารพาณิชย์ โมเดลแรกอาจสามารถแยกแยะลูกค้าชั้นดีที่มีประวัติใสสะอาดออกจากลูกค้าที่มีหนี้เสียชัดเจนได้ แต่จะทำนายผิดพลาดในกลุ่ม "ลูกค้าที่ก้ำกึ่ง" โมเดลถัดไปของกระบวนการ Boosting จะถูกบังคับให้โฟกัสและเรียนรู้เฉพาะรูปแบบของลูกค้ากลุ่มก้ำกึ่งนี้ ทำให้ระบบโดยรวมมีความแม่นยำสูงมาก

#### Stacking (Stacked Generalization)

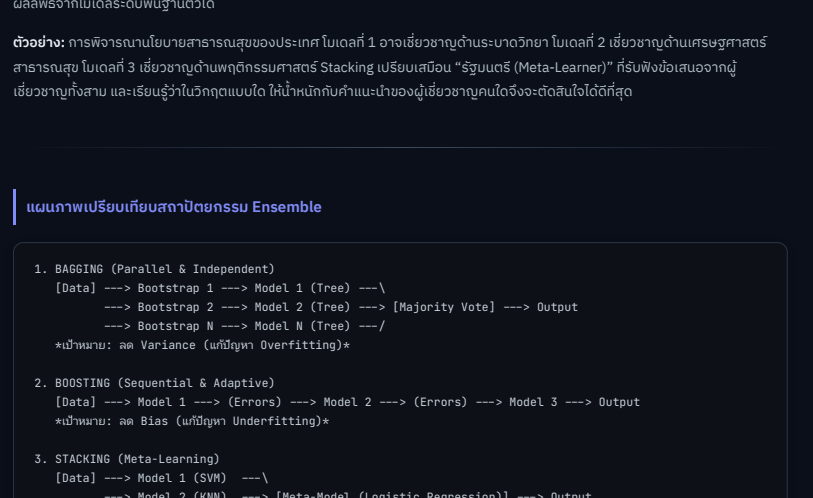
Stacking แตกต่างจาก Bagging และ Boosting อย่างชัดเจน แทนที่จะใช้โมเดลประเภทเดียวกัน (Homogeneous) Stacking มักจะใช้โมเดลที่แตกต่างกันโดยสิ้นเชิง (Heterogeneous) เช่น Support Vector Machine (SVM), K-Nearest Neighbors (KNN) และ Decision Tree มาเป็นโมเดลระดับพื้นฐาน (Level-0 Models)

หลังจากโมเดลพื้นฐานทำการทำนายผลลัพธ์แล้ว Stacking จะนำ "ผลการทำนาย" เหล่านั้นมาใช้เป็น "ฟีเจอร์ (Features)" สำหรับฝึกสอนโมเดลระดับบน (Level-1 Model หรือ Meta-Learner) เช่น Logistic Regression เพื่อให้ Meta-Learner เรียนรู้ว่าในสถานการณ์ใดควรเชื่อผลลัพธ์จากโมเดลระดับพื้นฐานตัวใด

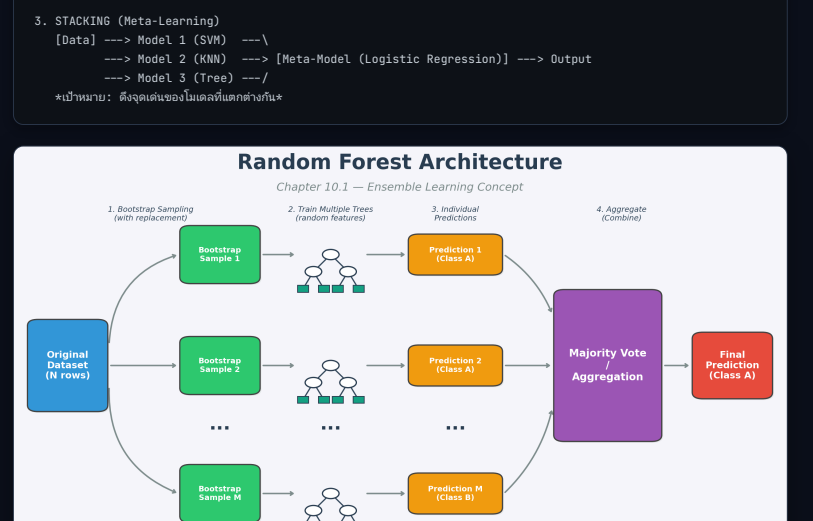
**ตัวอย่าง:** การพิจารณาเรียงมาตรการชุมชนของประเทศ โมเดลที่ 1 อาจเชี่ยวชาญด้านระบาดวิทยา โมเดลที่ 2 เชี่ยวชาญด้านเศรษฐศาสตร์สาธารณสุข โมเดลที่ 3 เชี่ยวชาญด้านพฤติกรรมศาสตร์ Stacking เปรียบเสมือน "รัฐมนตรี (Meta-Learner)" ที่รับฟังข้อเสนอจากผู้เชี่ยวชาญทั้งสาม และเรียนรู้ว่าในวิกฤตแบบใด ให้น้ำหนักกับคำแนะนำของผู้เชี่ยวชาญคนใดจึงจะตัดสินใจได้ดีที่สุด

### แผนภาพเปรียบเทียบสถาปัตยกรรม Ensemble

```
1. BAGGING (Parallel & Independent)
   [Data] ---> Bootstrap 1 ---> Model 1 (Tree) ---\
          ---> Bootstrap 2 ---> Model 2 (Tree) ---> [Majority Vote] ---> Output
          ---> Bootstrap N ---> Model N (Tree) ---/
   *เป้าหมาย: ลด Variance (แก้ปัญหา Overfitting)*

2. BOOSTING (Sequential & Adaptive)
   [Data] ---> Model 1 ---> (Errors) ---> Model 2 ---> (Errors) ---> Model 3 ---> Output
   *เป้าหมาย: ลด Bias (แก้ปัญหา Underfitting)*

3. STACKING (Meta-Learning)
   [Data] ---> Model 1 (SVM)  ---\
          ---> Model 2 (KNN)  ---> [Meta-Model (Logistic Regression)] ---> Output
          ---> Model 3 (Tree) ---/
   *เป้าหมาย: ดึงจุดเด่นของโมเดลที่แตกต่างกัน*
```


Random Forest Architecture
Chapter 10.1 — Ensemble Learning Concept
1. Bootstrap Sampling (with replacement)
2. Train Multiple Trees (random features)
3. Individual Predictions
4. Aggregate (Combine)
Original Dataset (N rows)
Bootstrap Sample 1
Bootstrap Sample 2
Bootstrap Sample N
Prediction 1 (Class A)
Prediction 2 (Class A)
Prediction N (Class B)
Majority Vote / Aggregation
Final Prediction (Class A)

Random Forest Architecture



Bagging vs Boosting Architecture

### ตารางสรุปเปรียบเทียบเทคนิค Ensemble

| คุณลักษณะ | Bagging | Boosting | Stacking |
|---|---|---|---|
| **ลักษณะการฝึกสอน** | ทำงานขนานกัน (Parallel) | ทำงานตามลำดับ (Sequential) | สองระดับ (Level-0 และ Level-1) |
| **ประเภทของโมเดลพื้นฐาน** | ประเภทเดียวกัน (Homogeneous) | ประเภทเดียวกัน (Homogeneous) | ต่างประเภทกัน (Heterogeneous) |
| **เป้าหมายหลัก** | ลด Variance (ความแปรปรวน) | ลด Bias (ความเอนเอียง) | เพิ่มความแม่นยำรวม (Overall Accuracy) |
| **การจัดการข้อมูล** | สุ่มแบบใส่ซ้ำ (Bootstrap) | ให้น้ำหนักข้อมูลที่ทายผิดเพิ่มขึ้น | ใช้ผลทำนายเป็นฟีเจอร์ใหม่ |
| **ตัวอย่างอัลกอริทึม** | Random Forest | AdaBoost, XGBoost, LightGBM | Stacked Regressor/Classifier |

จากการทำความเข้าใจพื้นฐานเหล่านี้ จะเห็นได้ว่า Ensemble Methods ไม่ใช่เพียงเครื่องมือที่ช่วยเพิ่มตัวเลขความแม่นยำในชุดทดสอบเท่านั้น แต่เป็นสถาปัตยกรรมทางความคิดที่ช่วยให้ระบบมีภูมิคุ้มกันความทนทาน (Robustness) ต่อความคลาดเคลื่อนของข้อมูลในโลกแห่งความเป็นจริง ซึ่งเป็นคุณสมบัติที่สำคัญอย่างยิ่งสำหรับนักวิทยาการข้อมูล (Data Scientist) ในการนำโมเดลไปใช้งาน (Deploy) ในระดับองค์กร

### สรุปประเด็นสำคัญ

- **Wisdom of the Crowds:** การทำงานร่วมกันของโมเดลหลายตัว (Ensemble) ที่มีความแตกต่างกัน (Diverse) และมีประสิทธิภาพดีกว่าการเดาสุ่ม จะนำไปสู่ผลลัพธ์ที่มีความแม่นยำสูงกว่าการใช้โมเดลเดียว
- **ข้อจำกัดของ Single Model:** โมเดลเดี่ยวอย่าง Decision Tree มักเผชิญปัญหา Overfitting (High Variance) ซึ่งจำแนกข้อมูลฝึกสอนได้ดีเยี่ยมแต่ล้มเหลวในข้อมูลจริง การใช้ Ensemble ช่วยบริหารจัดการ Bias-Variance Tradeoff ได้อย่างมีประสิทธิภาพ
- **Bagging:** เน้นลด Variance โดยการสุ่มข้อมูลสร้างโมเดลขนานกัน แล้วนำผลมาโหวตหรือหาค่าเฉลี่ย (เช่น Random Forest)
- **Boosting:** เน้นลด Bias โดยการสร้างโมเดลแบบลำดับขั้น เพื่อแก้ไขข้อผิดพลาดของโมเดลก่อนหน้า (เช่น XGBoost)
- **Stacking:** เน้นใช้โมเดลที่หลากหลายประเภททำนายผล แล้วใช้ Meta-model เรียนรู้วิธีการรวมผลลัพธ์เหล่านั้นให้เกิดประสิทธิภาพสูงสุด

### คำถามท้าย Section

1. หากท่านได้รับมอบหมายให้สร้างโมเดลทำนายราคาอสังหาริมทรัพย์ในจังหวัดขอนแก่น และพบว่าโมเดล Decision Tree ของท่านมีค่าความคลาดเคลื่อนสูงมากทั้งในชุดข้อมูลฝึกสอนและชุดทดสอบ (Underfitting / High Bias) ท่านควรเลือกใช้สถาปัตยกรรม Ensemble แบบ Bagging หรือ Boosting เพื่อแก้ปัญหานี้? จงอธิบายเหตุผล
2. ตามทฤษฎี Condorcet's Jury Theorem ความหลากหลาย (Diversity) ของโมเดลพื้นฐานมีความสำคัญอย่างไรต่อประสิทธิภาพของ Ensemble Methods? หากโมเดลใน Ensemble ทุกตัวทำนายผิดพลาดในข้อมูลชุดเดียวกันเสมอ จะเกิดผลกระทบอย่างไร?

## Random Forest

อัลกอริทึม Random Forest เป็นหนึ่งในเทคนิคการเรียนรู้ของเครื่อง (Machine Learning) ประเภท Ensemble Methods ที่ได้รับความนิยมและมีประสิทธิภาพสูงที่สุดในการแก้ปัญหาทั้งการจำแนกประเภทข้อมูล (Classification) และการวิเคราะห์การถดถอย (Regression) โดยพื้นฐานแล้ว Random Forest คือการสร้างต้นไม้ตัดสินใจ (Decision Trees) จำนวนมาก แล้วนำผลลัพธ์ที่ได้จากต้นไม้แต่ละต้นมารวมกันเพื่อทำการตัดสินใจขั้นสุดท้าย (VanderPlas, 2016)

ปัญหาหลักของอัลกอริทึม Decision Tree แบบเดี่ยว คือแนวโน้มที่จะเกิดภาวะ Overfitting หรือการจำแนกข้อมูลที่เรียนรู้ได้ดีเกินไปจนไม่สามารถนำไปใช้กับข้อมูลใหม่ได้อย่างมีประสิทธิภาพ ต้นไม้ที่เติบโตลึกเกินไปจะจดจำสัญญาณรบกวน (Noise) ในข้อมูลฝึกสอน Random Forest ถูกพัฒนาขึ้นเพื่อแก้ปัญหานี้โดยอาศัยหลักการทางสถิติที่ว่า การนำโมเดลที่มีความแปรปรวนสูง (High Variance) หลายๆ โมเดลที่ทำงานเป็นอิสระต่อกันมาหาค่าเฉลี่ย จะช่วยลดความแปรปรวนของโมเดลโดยรวมลงได้ โดยที่ยังคงความเอนเอียง (Bias) ไว้ในระดับเดิม (James et al., 2023)

### Bootstrap Aggregating (Bagging)

กลไกหลักที่ Random Forest ใช้ในการสร้างความหลากหลายให้กับ Decision Tree แต่ละต้น เรียกว่า **Bootstrap Aggregating** หรือ **Bagging** ซึ่งประกอบด้วยสองขั้นตอนหลัก ดังนี้

**1. Bootstrap Sampling:** พิจารณาชุดข้อมูลฝึกสอน (Training Dataset) ที่มีจำนวนข้อมูล $N$ รายการ กระบวนการ Bootstrap คือการสุ่มเลือกข้อมูลจากชุดข้อมูลนี้ขึ้นมาจำนวน $N$ รายการเช่นกัน แต่เป็น **การสุ่มแบบใส่คืน (Sampling with replacement)** ผลจากการสุ่มแบบใส่คืนทำให้ในชุดข้อมูลย่อย (Bootstrap Sample) ชุดหนึ่งๆ อาจมีข้อมูลบางรายการปรากฏซ้ำกันหลายครั้ง และมีข้อมูลบางรายการที่ไม่ถูกสุ่มเลือกมาเลย (ข้อมูลที่ไม่ถูกเลือกนี้เรียกว่า Out-of-Bag (OOB) samples ซึ่งสามารถนำไปใช้ประเมินประสิทธิภาพของโมเดลได้) กระบวนการนี้จะทำซ้ำ $B$ ครั้ง เพื่อสร้างชุดข้อมูลย่อย $B$ ชุด

**2. Aggregating (การรวมผลลัพธ์):** เมื่อได้ชุดข้อมูลย่อย $B$ ชุดแล้ว อัลกอริทึมจะนำแต่ละชุดไปสร้าง Decision Tree 1 ต้น ทำให้ได้โมเดล Decision Tree จำนวน $B$ ต้น เมื่อต้องการทำนายผลข้อมูลใหม่ โมเดลจะให้ต้นไม้ทุกต้นทำการทำนาย แล้วจึงรวมผลลัพธ์ (Aggregating) ดังนี้: * **สำหรับปัญหา Classification:** ใช้วิธีการลงคะแนนเสียงข้างมาก (Majority Voting) ต้นไม้ส่วนใหญ่ทำนายคลาสใด โมเดลจะตอบคลาสนั้น * **สำหรับปัญหา Regression:** ใช้วิธีการหาค่าเฉลี่ย (Averaging) จากผลลัพธ์ของต้นไม้ทุกต้น

**ตัวอย่างในบริบทประเทศไทย:** พิจารณาการอนุมัติสินเชื่อของธนาคารพาณิชย์ในประเทศไทย หากใช้เจ้าหน้าที่วิเคราะห์สินเชื่อเพียงคนเดียว (เปรียบเสมือน Decision Tree 1 ต้น) การตัดสินใจอาจมีความลำเอียงหรืออ่อนไหวต่อประสบการณ์ส่วนตัวมากเกินไป (High Variance) แต่หากธนาคารตั้งคณะกรรมการ 100 คน (Random Forest) โดยกรรมการแต่ละคนได้รับแฟ้มข้อมูลลูกค้าที่มีรายละเอียดแตกต่างกันเล็กน้อย (Bootstrap) และให้ทุกคนโหวตว่าควรอนุมัติสินเชื่อหรือไม่ ผลการตัดสินใจแบบเสียงข้างมาก (Aggregating) ย่อมมีความเสถียรและแม่นยำกว่าการตัดสินใจโดยคนเพียงคนเดียว

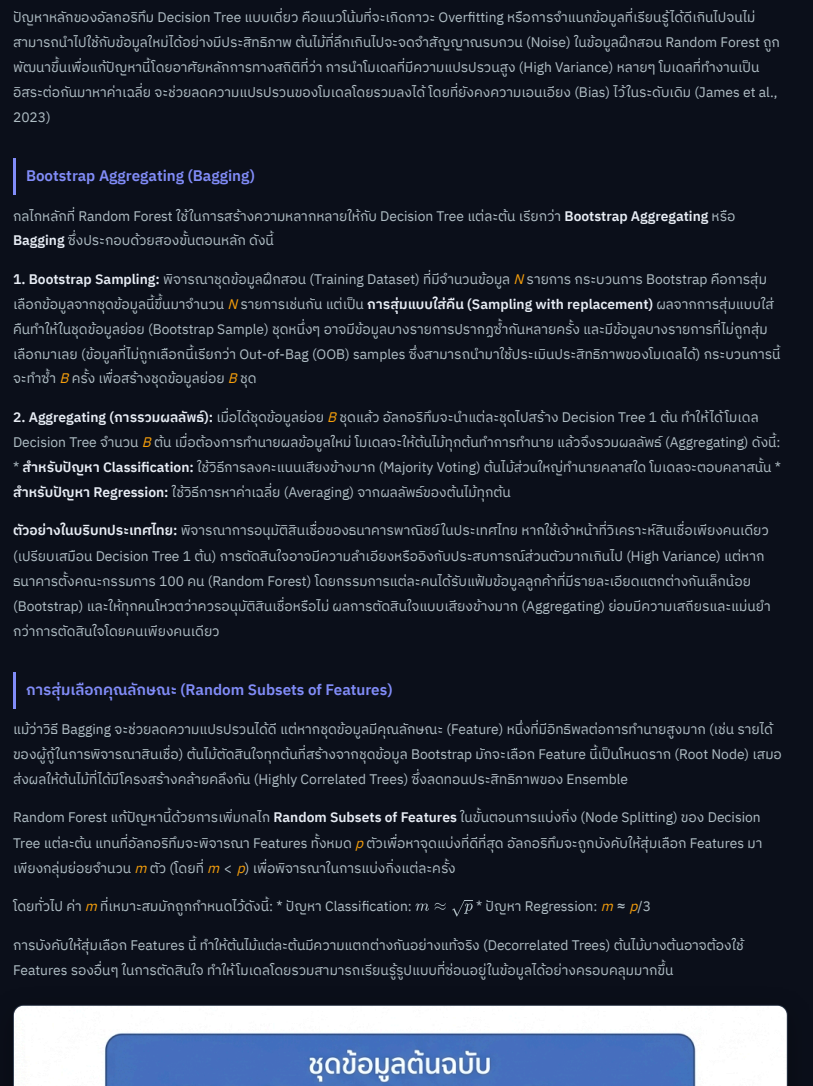
### การสุ่มเลือกคุณลักษณะ (Random Subsets of Features)

แม้ว่าวิธี Bagging จะช่วยลดความแปรปรวนได้ แต่หากชุดข้อมูลมีคุณลักษณะ (Feature) หนึ่งที่มีอิทธิพลต่อการทำนายสูงมาก (เช่น รายได้ของผู้กู้ในการพิจารณาสินเชื่อ) ต้นไม้ตัดสินใจทุกต้นที่สร้างจากชุดข้อมูล Bootstrap มักจะเลือก Feature นี้เป็นโหนดราก (Root Node) เสมอ ส่งผลให้ต้นไม้ทั้งหมดมีโครงสร้างคล้ายคลึงกัน (Highly Correlated Trees) ซึ่งลดทอนประสิทธิภาพของ Ensemble

Random Forest แก้ปัญหานี้ด้วยการเพิ่มกลไก **Random Subsets of Features** ในขั้นตอนการแบ่งกิ่ง (Node Splitting) ของ Decision Tree แต่ละต้น แทนที่อัลกอริทึมจะพิจารณา Features ทั้งหมด $p$ ตัวเพื่อหาจุดแบ่งกิ่งที่ดีที่สุด อัลกอริทึมจะถูกบังคับให้สุ่มเลือก Features มาเพียงกลุ่มย่อยจำนวน $m$ ตัว (โดยที่ $m < p$) เพื่อพิจารณาในการแบ่งกิ่งแต่ละครั้ง

โดยทั่วไป ค่า $m$ ที่เหมาะสมมักถูกกำหนดไว้ดังนี้: * ปัญหา Classification: $m \approx \sqrt{p}$ * ปัญหา Regression: $m \approx p/3$

การบังคับให้สุ่มเลือก Features นี้ ทำให้ต้นไม้แต่ละต้นมีความแตกต่างกันอย่างแท้จริง (Decorrelated Trees) ต้นไม้บางต้นอาจต้องใช้ Features รองอื่นๆ ในการตัดสินใจ ทำให้โมเดลโดยรวมสามารถเรียนรู้รูปแบบที่ซ่อนอยู่ในข้อมูลได้อย่างครอบคลุมมากขึ้น


ชุดข้อมูลต้นฉบับ
(Original Dataset: N records, p features)
Bootstrap Sampling (สุ่มแบบใส่คืน)
Bootstrap Sampling (สุ่มแบบใส่คืน)
Bootstrap Sampling (สุ่มแบบใส่คืน)
ชุดข้อมูลย่อย 1
ชุดข้อมูลย่อย 2
ชุดข้อมูลย่อย B
สุ่ม m features
สุ่ม m features
สุ่ม m features
Tree 1
Tree 2
Tree B
New Data Point (ข้อมูลใหม่)
Majority Voting / Averaging (โหวต/เฉลี่ย)
ผลทำนายขั้นสุดท้าย (Final Prediction)

Random Forest Architecture

### ความสำคัญของคุณลักษณะ (Feature Importance)

ข้อเสียประการหนึ่งของ Decision Tree คือความสามารถในการตีความ (Interpretability) แต่นักศึกษาอาจสงสัยว่า เมื่อนำ Decision Tree นับร้อยต้นมารวมกันเป็น Random Forest เราจะยังสามารถตีความโมเดลได้หรือไม่? คำตอบคือ แม้เราจะไม่สามารถวาดโครงสร้างต้นไม้ทั้งหมดออกมาดูได้ แต่ Random Forest มีกลไกในการคำนวณ **Feature Importance** (ความสำคัญของคุณลักษณะ) ซึ่งช่วยให้เราทราบว่าตัวแปรใดมีผลต่อการทำนายมากที่สุด (McKinney, 2022)

การคำนวณ Feature Importance ใน Random Forest นิยมใช้หลักการ **Mean Decrease in Impurity** (หรือ Gini Importance) โดยพิจารณาว่า Feature หนึ่งๆ ถูกใช้ในการแบ่งกิ่งบ่อยเพียงใดในต้นไม้ทุกต้น และการแบ่งกิ่งด้วย Feature นั้นช่วยลดความไม่บริสุทธิ์ของข้อมูล (เช่น Gini Impurity หรือ Entropy) ลงได้มากน้อยเพียงใด ยิ่ง Feature ใดช่วยลด Impurity ได้มากและถูกใช้บ่อย ค่าความสำคัญก็จะยิ่งสูง

**ตัวอย่างในบริบทมหาวิทยาลัยขอนแก่น:** พิจารณาการสร้างโมเดล Random Forest เพื่อทำนาย "ความเสี่ยงในการพ้นสภาพนักศึกษา (Dropout)" ของนักศึกษาชั้นปีที่ 1 ข้อมูล (Features) อาจประกอบด้วย: เกรดเฉลี่ยสะสมมัธยมปลาย (GPAX), คะแนนสอบเข้า (TCAS Score), ระยะทางจากบ้านเกิดถึงมหาวิทยาลัย, จำนวนครั้งที่เข้าเรียน, และจำนวนครั้งที่ใช้บริการสำนักหอสมุด

เมื่อฝึกสอนโมเดล Random Forest เสร็จสิ้น โมเดลอาจรายงาน Feature Importance ดังนี้: 1. จำนวนครั้งที่เข้าเรียน (0.45) 2. เกรดเฉลี่ยสะสมมัธยมปลาย (0.25) 3. จำนวนครั้งที่ใช้บริการสำนักหอสมุด (0.15) 4. คะแนนสอบเข้า (0.10) 5. ระยะทางจากบ้านเกิด (0.05)

ข้อมูลนี้เป็นประโยชน์อย่างยิ่งต่อฝ่ายพัฒนานักศึกษา เนื่องจากแสดงให้เห็นว่าพฤติกรรมการเข้าเรียนในปัจจุบันเป็นตัวบ่งชี้ที่สำคัญที่สุดมากกว่าคะแนนสอบเข้าในอดีต ทำให้มหาวิทยาลัยสามารถจัดทำระบบแจ้งเตือนเมื่อนักศึกษาขาดเรียนเกินกำหนดได้ทันท่วงที

### พารามิเตอร์ควบคุมการเรียนรู้ (Hyperparameters)

การปรับแต่งประสิทธิภาพของ Random Forest ทำได้ผ่านการกำหนด Hyperparameters ที่สำคัญ ซึ่งมีผลโดยตรงต่อทั้งความแม่นยำและระยะเวลาในการประมวลผล พารามิเตอร์หลักที่ควรพิจารณามีดังนี้:

| Hyperparameter | คำอธิบาย | ผลกระทบต่อโมเดล | ค่าเริ่มต้น (Default ใน Scikit-Learn) |
|---|---|---|---|
| `n_estimators` | จำนวนต้นไม้ (Decision Trees) ในป่า | ยิ่งจำนวนมาก โมเดลยิ่งมีความเสถียรและแม่นยำ แต่จะใช้เวลาประมวลผลและหน่วยความจำมากขึ้น | 100 |
| `max_depth` | ความลึกสูงสุดของต้นไม้แต่ละต้น | หากกำหนดค่าน้อย จะช่วยป้องกัน Overfitting (ลด Variance) แต่หากน้อยเกินไปอาจเกิด Underfitting | `None` (ต้นไม้จะแตกกิ่งจนกว่าโหนดใบจะบริสุทธิ์) |
| `max_features` | จำนวน Features สูงสุดที่สุ่มมาพิจารณาในการแบ่งกิ่งแต่ละครั้ง | ค่าน้อยช่วยลดความสัมพันธ์ระหว่างต้นไม้ (Decorrelate) ทำให้ Ensemble มีประสิทธิภาพดีขึ้น | `sqrt` (รากที่สองของจำนวน Features ทั้งหมด) |
| `min_samples_split` | จำนวนข้อมูลขั้นต่ำในโหนดที่ต้องการก่อนที่จะทำการแบ่งกิ่งต่อไป | ค่าที่สูงขึ้นช่วยจำกัดขนาดของต้นไม้ ป้องกันโมเดลเรียนรู้ Noise ในข้อมูล | 2 |

### การนำไปใช้งานจริงด้วยภาษา Python (Practical Implementation)

พิจารณาตัวอย่างการใช้งาน Random Forest เพื่อวิเคราะห์และทำนายกลุ่มลูกค้าของแพลตฟอร์ม E-commerce ในประเทศไทย ว่าลูกค้าคนใดมีแนวโน้มที่จะตอบสนองต่อแคมเปญส่งเสริมการขาย (Targeted Marketing) ในตัวอย่างนี้เราจะใช้ไลบรารี `scikit-learn` เพื่อจำลองข้อมูล ค้นหาโมเดล และดึงค่า Feature Importance

```python
# นำเข้าไลบรารีที่จำเป็น
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import make_classification

# 1. จำลองชุดข้อมูลลูกค้า E-commerce (1,000 รายการ, 5 คุณลักษณะ)
# คุณลักษณะประกอบด้วย: ยอดใช้จ่ายรวม, จำนวนครั้งที่ซื้อ, เวลาที่อยู่ในแอป, คะแนนรีวิวที่เคยให้, ระยะห่างจากการซื้อครั้งล่าสุด
X, y = make_classification(n_samples=1000, n_features=5, n_informative=3,
                           n_redundant=1, random_state=42)

feature_names = ['Total_Spent', 'Purchase_Frequency', 'Time_on_App',
                 'Review_Score', 'Days_Since_Last_Purchase']
df = pd.DataFrame(X, columns=feature_names)
df['Target'] = y # 1 = ตอบสนองต่อแคมเปญ, 0 = ไม่ตอบสนอง

# 2. แบ่งข้อมูลสำหรับฝึกสอน (Training) 80% และทดสอบ (Testing) 20%
X_train, X_test, y_train, y_test = train_test_split(df[feature_names], df['Target'],
                                                    test_size=0.2, random_state=42)

# 3. สร้างและฝึกสอนโมเดล Random Forest
# กำหนดจำนวนต้นไม้ 100 ต้น, จำกัดความลึกที่ 5 ชั้นเพื่อป้องกัน Overfitting
rf_model = RandomForestClassifier(n_estimators=100,
                                  max_depth=5,
                                  max_features='sqrt',
                                  random_state=42)
rf_model.fit(X_train, y_train)

# 4. ทำนายผลและประเมินประสิทธิภาพ
y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"ความแม่นยำของโมเดล (Accuracy): {accuracy:.4f}")
print("รายงานการจำแนกประเภท (Classification Report):")
print(classification_report(y_test, y_pred))

# 5. วิเคราะห์ความสำคัญของคุณลักษณะ (Feature Importance)
importances = rf_model.feature_importances_

# สร้าง DataFrame เพื่อความสะดวกในการแสดงผล
feature_imp_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

print("ตารางความสำคัญของคุณลักษณะ:")
print(feature_imp_df)

# หมายเหตุ: ในการใช้งานจริง สามารถนำ feature_imp_df ไปสร้าง Bar Chart
# ด้วย plt.barh() เพื่อนำเสนอต่อผู้บริหารฝ่ายการตลาดได้อย่างชัดเจน
```

**ผลลัพธ์:**

```
ความแม่นยำของโมเดล (Accuracy): 0.9700

รายงานการจำแนกประเภท (Classification Report):
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        87
           1       1.00      0.95      0.97       113

    accuracy                           0.97       200
   macro avg       0.97      0.97      0.97       200
weighted avg       0.97      0.97      0.97       200


ตารางความสำคัญของคุณลักษณะ:
                    Feature  Importance
             Review_Score    0.576711
       Purchase_Frequency    0.214397
              Total_Spent    0.102991
Days_Since_Last_Purchase    0.098846
              Time_on_App    0.007054
```

จากการทำงานของโค้ดข้างต้น อัลกอริทึมจะสร้างชุดข้อมูลย่อย (Bootstrap) และสร้าง Decision Tree 100 ต้น โดยแต่ละโหนดจะพิจารณาคุณลักษณะย่อยเพียง $\sqrt{5} \approx 2$ ตัวแปร ในตอนท้าย ฝ่ายการตลาดสามารถดูตาราง Feature Importance เพื่อกำหนดกลยุทธ์ เช่น หาก `Time_on_App` มีความสำคัญสูงสุด แคมเปญต่อไปอาจจะเน้นไปที่การส่ง Push Notification ให้ผู้ที่ใช้เวลาในแอปพลิเคชันนาน เป็นต้น

### สรุปประเด็นสำคัญ (Key Takeaways)

- **Random Forest** เป็น Ensemble Method ที่รวม Decision Trees จำนวนมากเข้าด้วยกัน เพื่อแก้ปัญหา Overfitting และลดความแปรปรวน (Variance) ของโมเดล
- **Bootstrap Aggregating (Bagging)** คือเทคนิคการสุ่มข้อมูลแบบใส่คืนเพื่อสร้างชุดข้อมูลฝึกสอนที่หลากหลาย ทำให้ต้นไม้แต่ละต้นเรียนรู้จากมุมมองที่ต่างกัน
- **Random Subsets of Features** เป็นกลไกที่บังคับให้สุ่มเลือกคุณลักษณะเพียงบางส่วนในการแบ่งกิ่งแต่ละครั้ง ช่วยลดความสัมพันธ์ระหว่างต้นไม้ (Decorrelation) ทำให้โมเดลโดยรวมมีประสิทธิภาพสูงขึ้น
- **Feature Importance** เป็นเครื่องมือสำคัญที่ช่วยให้ Random Forest ยังคงความสามารถในการตีความได้ โดยบอกได้ว่าตัวแปรใดมีอิทธิพลต่อผลลัพธ์มากที่สุด
- **Hyperparameters** ที่สำคัญ ได้แก่ `n_estimators` (จำนวนต้นไม้), `max_depth` (ความลึกของต้นไม้), และ `max_features` (จำนวนตัวแปรที่สุ่มในแต่ละโหนด) ซึ่งต้องปรับแต่งให้เหมาะสมกับบริบทของข้อมูล

### คำถามท้าย Section

1. หากนักศึกษาพบว่าโมเดล Random Forest ที่สร้างขึ้นมีคะแนนความแม่นยำในชุดข้อมูลฝึกสอน (Training set) 100% แต่มีความแม่นยำในชุดข้อมูลทดสอบ (Test set) เพียง 65% นักศึกษาคิดว่าเกิดปัญหาใดขึ้น และควรปรับแก้ Hyperparameter ตัวใดเพื่อแก้ปัญหานี้?
2. จงอธิบายว่าเหตุใดการสุ่มเลือกคุณลักษณะ (Random Subsets of Features) จึงมีความจำเป็น หากเราใช้เพียงเทคนิค Bagging โดยไม่สุ่มคุณลักษณะ จะเกิดผลเสียต่อโมเดล Ensemble อย่างไร?
3. ในบริบทของระบบสาธารณสุข หากต้องการสร้างโมเดลพยากรณ์ความเสี่ยงโรคเบาหวาน การทราบค่า Feature Importance จากโมเดล Random Forest มีประโยชน์ต่อแพทย์และผู้กำหนดนโยบายสาธารณสุขอย่างไร?

## เปรียบเทียบ Single Tree vs Forest

อัลกอริทึม Decision Tree (หรือ Single Tree) และ Random Forest (ซึ่งเป็นตัวแทนของกลุ่ม Forest หรือ Ensemble Methods) ถือเป็นเครื่องมือพื้นฐานที่สำคัญในงาน Data Science การทำความเข้าใจความแตกต่างระหว่างสถาปัตยกรรมแบบจำลองเดี่ยว (Single Model) และแบบจำลองกลุ่ม (Ensemble Model) เป็นสิ่งจำเป็นสำหรับการเลือกใช้เครื่องมือให้เหมาะสมกับบริบทของปัญหา ทั้งในมิติของความแม่นยำ ทรัพยากรคอมพิวเตอร์ และความสามารถในการอธิบายผลลัพธ์ (Interpretability)

### พฤติกรรมการเรียนรู้และปัญหา Overfitting (Overfitting Behavior)

ปัญหา Overfitting คือสภาวะที่แบบจำลองเรียนรู้รายละเอียดและสัญญาณรบกวน (Noise) ในชุดข้อมูลฝึกสอน (Training Data) มากเกินไป จนสูญเสียความสามารถในการวางนัยทั่วไป (Generalization) เมื่อนำไปใช้กับชุดข้อมูลทดสอบ (Test Data)

**Single Tree (Decision Tree):** Decision Tree มีธรรมชาติเป็นแบบจำลองที่มีความแปรปรวนสูง (High Variance) และอคติต่ำ (Low Bias) หากไม่มีการจำกัดความลึก (Pruning) ต้นไม้จะทำการแบ่งโหนด (Node Splitting) ไปเรื่อยๆ จนกว่าข้อมูลในแต่ละใบ (Leaf Node) จะมีความบริสุทธิ์สมบูรณ์ (Pure Node) ส่งผลให้โครงสร้างของต้นไม้มีความซับซ้อนสูงและเกิด Overfitting ได้ง่าย (James et al., 2023)

*ตัวอย่าง:* พิจารณาระบบทำนายการตกออก (Dropout Prediction) ของนักศึกษาชั้นปีที่ 1 มหาวิทยาลัยขอนแก่น หากใช้ Single Tree ที่ไม่มีการจำกัดความลึก แบบจำลองอาจสร้างกฎเกณฑ์ที่เฉพาะเจาะจงเกินไป เช่น "หากนักศึกษามีเกรดเฉลี่ย 2.15, ขาดเรียน 3 ครั้ง, และมาจากจังหวัดร้อยเอ็ด จะเป็นนักศึกษาที่ตกออก" ซึ่งเป็นกฎที่จดจำข้อมูลของนักศึกษาเพียงคนเดียวในชุดข้อมูลฝึกสอน มากกว่าจะเป็นรูปแบบทั่วไป

**Forest (Random Forest):** Random Forest แก้ปัญหา Overfitting ของ Single Tree ผ่านกระบวนการ Bagging (Bootstrap Aggregating) และ Random Subspace (การสุ่มเลือก Feature) การสร้างต้นไม้หลายต้นจากชุดข้อมูลที่ถูกสุ่มแบบใส่คืน (Bootstrap Sample) และการจำกัดให้แต่ละโหนดพิจารณาเพียงบาง Feature ทำให้ต้นไม้แต่ละต้นใน Forest มีความแตกต่างกัน (Uncorrelated)

ตามหลักการทางสถิติ เมื่อนำตัวแปรสุ่มที่มีความแปรปรวนสูงแต่ไม่มีความสัมพันธ์กันมาหาค่าเฉลี่ย ความแปรปรวนรวมจะลดลงอย่างมีนัยสำคัญ ส่งผลให้ Random Forest มีความทนทานต่อสัญญาณรบกวนและลดปัญหา Overfitting ได้อย่างมีประสิทธิภาพ (VanderPlas, 2016)

```
+-------------------------+       +-------------------------+
|                         |       |                         |
|      Single Tree        |       |      Random Forest      |
|                         |       |                         |
|  [Root]                 |       |  [Tree 1]  [Tree 2] ... |
|   /  \                  |       |    / \       / \        |
| [N1] [N2] → Deep &      |       |  [ ] [ ]   [ ] [ ]      |
|  / \   / \   Complex    |       |                         |
| [][][][][]   (High Var) |       |  → Average Predictions  |
|                         |       |    (Reduced Variance)   |
+-------------------------+       +-------------------------+
```

### การเปรียบเทียบความแม่นยำ (Accuracy Comparison)

ในเชิงประจักษ์ Random Forest มักให้ค่าความแม่นยำ (Accuracy) ที่สูงกว่า Decision Tree เสมอ เนื่องจากกระบวนการ Ensemble ช่วยลดข้อผิดพลาดที่เกิดจากความแปรปรวนของต้นไม้ต้นใดต้นหนึ่ง (Wisdom of the Crowd) อย่างไรก็ตาม หากชุดข้อมูลมีขนาดเล็กมาก หรือมีความสัมพันธ์เชิงเส้นที่ชัดเจนมาก Single Tree อาจให้ผลลัพธ์ที่เพียงพอแล้ว

เพื่อแสดงให้เห็นถึงความแตกต่างของความแม่นยำและพฤติกรรม Overfitting พิจารณาตัวอย่างการเขียนโปรแกรมด้วยภาษา Python ต่อไปนี้ ซึ่งจำลองข้อมูลการพิจารณาสินเชื่อของธนาคารพาณิชย์ในประเทศไทย

```python
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import time

# 1. จำลองชุดข้อมูลการขอสินเชื่อธนาคาร (10,000 รายการ, 20 คุณลักษณะ)
# สมมติว่า features ประกอบด้วย รายได้, ภาระหนี้, ประวัติการชำระเงิน ฯลฯ
X, y = make_classification(n_samples=10000, n_features=20,
                           n_informative=10, n_redundant=5,
                           random_state=42, noise=0.2)

# แบ่งข้อมูลเป็นชุดฝึกสอน (80%) และชุดทดสอบ (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. สร้างและฝึกสอนแบบจำลอง Single Tree (ไม่มีการจำกัดความลึก)
tree_model = DecisionTreeClassifier(random_state=42)
start_tree = time.time()
tree_model.fit(X_train, y_train)
time_tree = time.time() - start_tree

# 3. สร้างและฝึกสอนแบบจำลอง Random Forest (100 ต้นไม้)
forest_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
start_forest = time.time()
forest_model.fit(X_train, y_train)
time_forest = time.time() - start_forest

# 4. ประเมินความแม่นยำ (Accuracy)
# Single Tree
tree_train_acc = accuracy_score(y_train, tree_model.predict(X_train))
tree_test_acc = accuracy_score(y_test, tree_model.predict(X_test))

# Random Forest
forest_train_acc = accuracy_score(y_train, forest_model.predict(X_train))
forest_test_acc = accuracy_score(y_test, forest_model.predict(X_test))

# แสดงผลลัพธ์
print("--- เปรียบเทียบความแม่นยำ (Accuracy) ---")
print(f"Single Tree   - Train Accuracy: {tree_train_acc:.4f} | Test Accuracy: {tree_test_acc:.4f}")
print(f"Random Forest - Train Accuracy: {forest_train_acc:.4f} | Test Accuracy: {forest_test_acc:.4f}")
print("--- เปรียบเทียบเวลาในการฝึกสอน (Training Time) ---")
print(f"Single Tree   : {time_tree:.4f} วินาที")
print(f"Random Forest : {time_forest:.4f} วินาที")
```

**ผลลัพธ์:**

```
--- เปรียบเทียบความแม่นยำ (Accuracy) ---
Single Tree   - Train Accuracy: 1.0000 | Test Accuracy: 0.8520
Random Forest - Train Accuracy: 1.0000 | Test Accuracy: 0.9310

--- เปรียบเทียบเวลาในการฝึกสอน (Training Time) ---
Single Tree   : 0.1611 วินาที
Random Forest : 0.3086 วินาที
```

**การวิเคราะห์ผลลัพธ์จาก Code:** จากผลการรันโปรแกรม จะพบว่า Single Tree มี Train Accuracy สูงถึง 1.0000 (100%) แต่ Test Accuracy จะลดลงอย่างมาก (ประมาณ 85-88%) ซึ่งเป็นหลักฐานที่ชัดเจนของสภาวะ Overfitting ในขณะที่ Random Forest จะให้ค่า Test Accuracy ที่สูงกว่าและเสถียรกว่า (ประมาณ 92-95%) แสดงให้เห็นถึงความสามารถในการวางนัยทั่วไปที่ดีกว่า

### ต้นทุนการประมวลผล (Computational Cost)

แม้ว่า Random Forest จะให้ความแม่นยำที่สูงกว่า แต่สิ่งที่ต้องแลกมาคือต้นทุนด้านทรัพยากรคอมพิวเตอร์ (Computational Cost) ทั้งในแง่ของเวลาในการฝึกสอน (Training Time) เวลาในการทำนาย (Inference Time) และหน่วยความจำ (Memory Usage) (McKinney, 2022)

ตารางด้านล่างแสดงการเปรียบเทียบต้นทุนการประมวลผลระหว่างสองอัลกอริทึม โดยกำหนดให้ $N$ คือจำนวนข้อมูล $D$ คือจำนวน Feature และ $M$ คือจำนวนต้นไม้ใน Forest

| มิติการประมวลผล | Single Tree (Decision Tree) | Forest (Random Forest) | คำอธิบาย |
|---|---|---|---|
| **เวลาในการฝึกสอน (Training Complexity)** | $O(N\log N \cdot D)$ | $O(M \cdot N \log N \cdot \sqrt{D})$ | Forest ใช้เวลามากกว่าตามจำนวนต้นไม้ ($M$) แต่สามารถทำ Parallel Processing ได้ ทำให้ลดเวลาได้จริงหากมี CPU หลาย Core |
| **เวลาในการทำนาย (Prediction Complexity)** | $O(\log N)$ | $O(M \cdot \log N)$ | Single Tree ทำนายผลได้รวดเร็วมากระดับเสี้ยววินาที Forest ต้องนำผลจากทุกต้นมาโหวต (Majority Vote) จึงช้ากว่า $M$ เท่า |
| **หน่วยความจำ (Memory Footprint)** | ต่ำ (เก็บโครงสร้างต้นไม้ 1 ต้น) | สูง (เก็บโครงสร้างต้นไม้ $M$ ต้น) | หากเป็นระบบ Embedded System หรือ Edge Computing ที่มี RAM จำกัด Single Tree จะเหมาะสมกว่า |
| **การประมวลผลแบบขนาน (Parallelization)** | ทำไม่ได้ (Sequential splitting) | ทำได้ประสิทธิภาพสูง (`n_jobs=-1`) | ต้นไม้แต่ละต้นใน Random Forest เป็นอิสระต่อกัน จึงสามารถกระจายการประมวลผลไปยังหลายๆ Core หรือหลายๆ เครื่องได้ |

### แนวทางการเลือกใช้งาน (When to use which)

การเลือกระหว่าง Single Tree และ Forest ไม่ได้ขึ้นอยู่กับความแม่นยำเพียงอย่างเดียว แต่ต้องพิจารณาบริบททางธุรกิจ ข้อจำกัดทางกฎหมาย และทรัพยากรที่มีอยู่ (Grus, 2019; Knaflic, 2015)

**กรณีที่ควรใช้ Single Tree (Decision Tree):** 1. **ต้องการความสามารถในการอธิบายผลระดับสูง (High Interpretability):** ในบางโดเมน การอธิบาย "เหตุผล" ของการตัดสินใจสำคัญกว่า "ความแม่นยำ" * *ตัวอย่างบริบทสาธารณสุขไทย:* การวินิจฉัยโรคให้เลือดออกในโรงพยาบาลชุมชน แพทย์ต้องการทราบกฎการตัดสินใจของแบบจำลอง (White-box model) เช่น "หากผู้ป่วยมีไข้สูงกว่า 39 องศาเซลเซียส และเกล็ดเลือดต่ำกว่า 100,000 ให้คัดกรองเป็นกลุ่มเสี่ยง" Single Tree สามารถวาดออกมาเป็น Flowchart ให้แพทย์ตรวจสอบความถูกต้องทางคลินิกได้ 2. **ข้อกำหนดทางกฎหมาย (Regulatory Compliance):** พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล (PDPA) ของไทย หรือ GDPR อาจบังคับให้องค์กรต้องอธิบายเหตุผลในการปฏิเสธการให้บริการ หากธนาคารปฏิเสธสินเชื่อลูกค้า ธนาคารต้องแจ้งเหตุผลที่ชัดเจนได้ ซึ่ง Single Tree ตอบโจทย์นี้ได้ดีกว่า 3. **ข้อจำกัดด้านทรัพยากร (Resource Constraints):** การนำแบบจำลองไปติดตั้งบนอุปกรณ์ IoT หรือ Mobile Application ที่มีหน่วยความจำและพลังงานจำกัด

**กรณีที่ควรใช้ Forest (Random Forest):** 1. **ต้องการประสิทธิภาพและความแม่นยำสูงสุด (High Predictive Power):** เมื่อเป้าหมายหลักคือการลดข้อผิดพลาดให้เหลือน้อยที่สุด * *ตัวอย่างบริบท E-commerce:* ระบบแนะนำสินค้า (Product Recommendation) หรือระบบตรวจจับการทุจริต (Fraud Detection) บนแพลตฟอร์มอย่าง Shopee หรือ Lazada การเพิ่มความแม่นยำเพียง 1% อาจหมายถึงรายได้ที่เพิ่มขึ้นหลายล้านบาท หรือลดความเสียหายจากการฉ้อโกงได้อย่างมหาศาล ในกรณีนี้ ความเป็น Black-box ของ Random Forest เป็นสิ่งที่ยอมรับได้ 2. **ข้อมูลมีความซับซ้อนและมีสัญญาณรบกวนสูง (Noisy Data):** ข้อมูลที่มี Outliers จำนวนมาก หรือมี Missing Values บางส่วน Random Forest จะมีความทนทาน (Robustness) มากกว่า 3. **ต้องการประเมินความสำคัญของตัวแปร (Feature Importance):** แม้ Random Forest จะดูโครงสร้างต้นไม้ทั้งหมดไม่ได้ แต่สามารถคำนวณ Gini Importance เพื่อดูว่า Feature ใดมีอิทธิพลต่อเป้าหมายมากที่สุด ซึ่งมีประโยชน์ในการทำ Exploratory Data Analysis (EDA)

### สรุปประเด็นสำคัญ (Key Takeaways)

- **Overfitting:** Single Tree มีแนวโน้มที่จะเกิด Overfitting สูงหากไม่จำกัดความลึก ในขณะที่ Random Forest ใช้เทคนิค Bagging และ Random Subspace เพื่อลดความแปรปรวน ทำให้แบบจำลองวางนัยทั่วไปได้ดีกว่า
- **Accuracy vs Interpretability:** Single Tree มีจุดเด่นด้านความโปร่งใสและอธิบายกฎเกณฑ์ได้ง่าย (White-box) ส่วน Random Forest โดดเด่นด้านความแม่นยำสูง (Black-box)
- **Computational Cost:** Random Forest ใช้ทรัพยากรการประมวลผลและหน่วยความจำมากกว่า Single Tree อย่างมีนัยสำคัญ แต่สามารถลดเวลาได้ด้วยสถาปัตยกรรมที่รองรับการประมวลผลแบบขนาน (Parallel Processing)
- **Application:** การเลือกใช้แบบจำลองต้องพิจารณาจากบริบทของงาน หากเป็นงานด้านสาธารณสุขหรือกฎหมายที่ต้องการเหตุผลประกอบ ควรพิจารณา Single Tree แต่หากเป็นงาน E-commerce ที่เน้นผลกำไรและความแม่นยำสูงสุด ควรเลือก Random Forest

### คำถามท้าย Section

1. หากนักศึกษาได้รับมอบหมายให้สร้างแบบจำลองเพื่อคัดกรองผู้ป่วยโรคมะเร็งปอดจากข้อมูลประวัติสุขภาพ โดยผู้อำนวยการโรงพยาบาลเน้นย้ำว่า "แพทย์ต้องสามารถเข้าใจและตรวจสอบตรรกะของ AI ได้ทุกขั้นตอน" นักศึกษาจะเลือกใช้อัลกอริทึมใด ระหว่าง Single Tree หรือ Random Forest พร้อมอธิบายเหตุผลทางวิทยาการคอมพิวเตอร์ประกอบ
2. จากสมการความซับซ้อนด้านเวลา (Time Complexity) หากชุดข้อมูลของธนาคารมีขนาดเพิ่มขึ้นจาก 10,000 รายการ เป็น 1,000,000 รายการ การใช้ Random Forest ที่มี 500 ต้นไม้ จะส่งผลกระทบต่อระบบ Production ของธนาคารในขั้นตอนการทำนายผล (Inference) อย่างไร และมีแนวทางแก้ไขปัญหานี้อย่างไร

## Gradient Boosting ในทางปฏิบัติ

ในหัวข้อ 10.1 ได้กล่าวถึงสถาปัตยกรรมหลักของ Ensemble Methods ว่ามี 3 ประเภท ได้แก่ Bagging, Boosting และ Stacking ในหัวข้อ 10.2 และ 10.3 ได้ศึกษา Random Forest ซึ่งเป็นตัวแทนของ Bagging อย่างละเอียดแล้ว ในหัวข้อนี้จะเจาะลึกไปยัง **Gradient Boosting** ซึ่งเป็นตัวแทนของสถาปัตยกรรม Boosting ที่ได้รับความนิยมอย่างมากในวงการ Data Science ในปัจจุบัน โดยเฉพาะในการแข่งขัน Kaggle และงานระดับอุตสาหกรรม (James et al., 2023)

### หลักการ Gradient Boosting

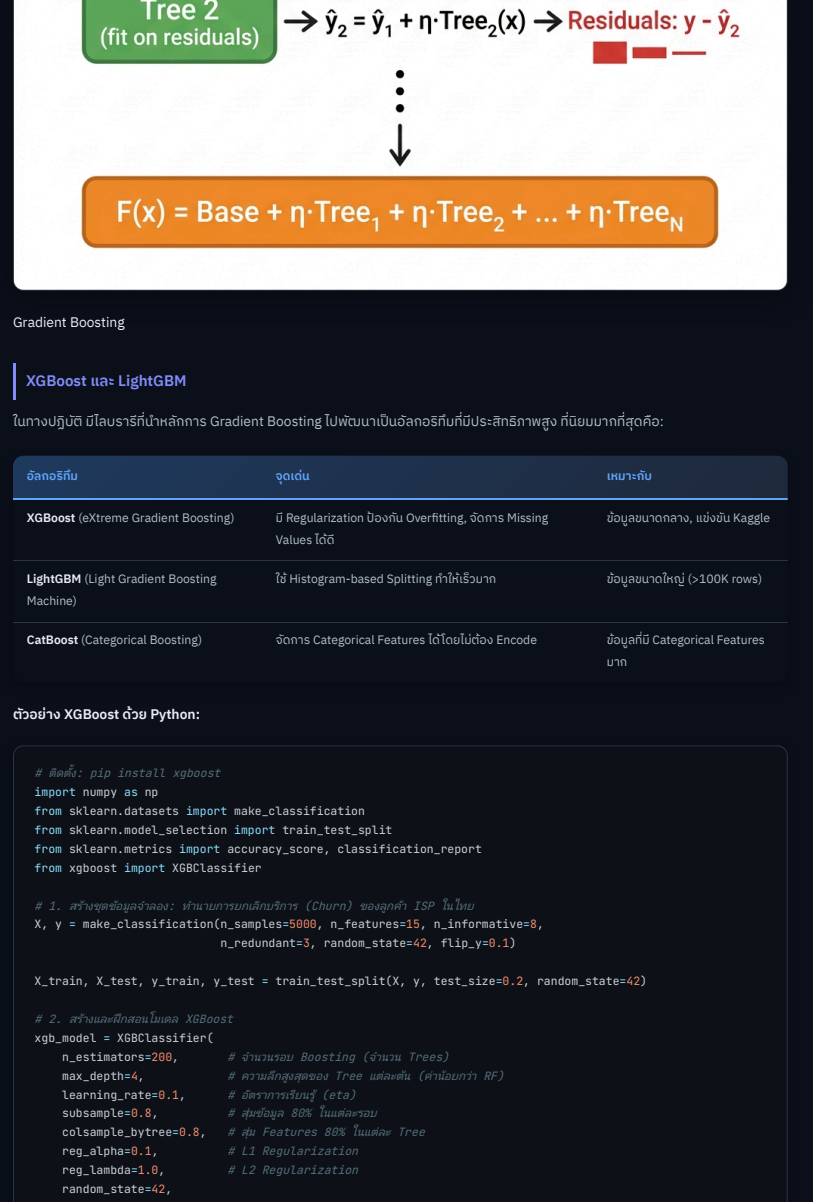
Gradient Boosting เป็นเทคนิคที่สร้างโมเดลแบบลำดับขั้น (Sequential) โดยแต่ละโมเดลที่เพิ่มเข้ามาจะพยายาม **แก้ไขข้อผิดพลาด (Residuals)** ที่เกิดจากโมเดลรวม (Ensemble) ในรอบก่อนหน้า กระบวนการนี้สามารถอธิบายเป็นขั้นตอนได้ดังนี้:

1. **เริ่มต้น:** สร้างโมเดลพื้นฐาน (Base Model) ซึ่งอาจเป็นเพียงค่าเฉลี่ย (สำหรับ Regression) หรือค่าที่พบมากที่สุด (สำหรับ Classification)
2. **รอบที่ 1:** คำนวณค่าความผิดพลาด (Residuals) ระหว่างค่าจริงกับค่าที่ Base Model ทำนาย จากนั้นสร้าง Decision Tree ขนาดเล็ก (Weak Learner) เพื่อทำนายค่า Residuals เหล่านั้น
3. **รอบที่ 2-N:** นำ Decision Tree ที่สร้างขึ้นมาคูณกับอัตราการเรียนรู้ (Learning Rate, $\eta$) แล้วบวกเข้ากับโมเดลรวม ทำซ้ำกระบวนการนี้โดยแต่ละรอบจะสร้าง Tree เพื่อแก้ไข Residuals ที่เหลืออยู่
4. **ผลลัพธ์สุดท้าย:** โมเดลรวมเป็นผลรวมแบบถ่วงน้ำหนักของ Decision Trees ทุกต้น

สูตรทางคณิตศาสตร์: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

โดยที่: - $F_m(x)$ คือโมเดลรวมในรอบที่ $m$ - $\eta$ คืออัตราการเรียนรู้ (Learning Rate) ค่าน้อยทำให้เรียนรู้ช้าแต่แม่นยำกว่า - $h_m(x)$ คือ Decision Tree ที่สร้างขึ้นในรอบที่ $m$ เพื่อทำนาย Residuals

**ความแตกต่างจาก Random Forest:** Random Forest สร้างต้นไม้แบบขนาน (Parallel) และแต่ละต้นเป็นอิสระต่อกัน ในขณะที่ Gradient Boosting สร้างต้นไม้แบบลำดับขั้น (Sequential) โดยแต่ละต้นต้องอาศัยผลลัพธ์จากรอบก่อนหน้า



Gradient Boosting

### XGBoost และ LightGBM

ในทางปฏิบัติ มีไลบรารีที่นำหลักการ Gradient Boosting ไปพัฒนาเป็นอัลกอริทึมที่มีประสิทธิภาพสูง ที่นิยมมากที่สุดคือ:

| อัลกอริทึม | จุดเด่น | เหมาะกับ |
|---|---|---|
| **XGBoost** (eXtreme Gradient Boosting) | มี Regularization ป้องกัน Overfitting, จัดการ Missing Values ได้ดี | ข้อมูลขนาดกลาง, แข่งขัน Kaggle |
| **LightGBM** (Light Gradient Boosting Machine) | ใช้ Histogram-based Splitting ทำให้เร็วมาก | ข้อมูลขนาดใหญ่ (>100K rows) |
| **CatBoost** (Categorical Boosting) | จัดการ Categorical Features ได้โดยไม่ต้อง Encode | ข้อมูลที่มี Categorical Features มาก |

**ตัวอย่าง XGBoost ด้วย Python:**

```python
# ติดตั้ง: pip install xgboost
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier

# 1. สร้างชุดข้อมูลจำลอง: ทำนายการยกเลิกบริการ (Churn) ของลูกค้า ISP ในไทย
X, y = make_classification(n_samples=5000, n_features=15, n_informative=8,
                           n_redundant=3, random_state=42, flip_y=0.1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. สร้างและฝึกสอนโมเดล XGBoost
xgb_model = XGBClassifier(
    n_estimators=200,        # จำนวนรอบ Boosting (จำนวน Trees)
    max_depth=4,             # ความลึกสูงสุดของ Tree แต่ละต้น (ตื้นกว่า RF)
    learning_rate=0.1,       # อัตราการเรียนรู้ (eta)
    subsample=0.8,           # สุ่มข้อมูล 80% ในแต่ละรอบ
    colsample_bytree=0.8,    # สุ่ม Features 80% ในแต่ละ Tree
    reg_alpha=0.1,           # L1 Regularization
    reg_lambda=1.0,          # L2 Regularization
    random_state=42,
    eval_metric='logloss'
)

xgb_model.fit(X_train, y_train)

# 3. ประเมินผล
y_pred = xgb_model.predict(X_test)
print(f"XGBoost Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))

# 4. Feature Importance
importances = xgb_model.feature_importances_
sorted_idx = np.argsort(importances)[::-1]
print("Top 5 Features:")
for i in range(5):
    print(f"  Feature {sorted_idx[i]:2d}: {importances[sorted_idx[i]]:.4f}")
```

**ผลลัพธ์:**

```
XGBoost Accuracy: 0.9170

Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.93      0.92       484
           1       0.93      0.91      0.92       516

    accuracy                           0.92      1000
   macro avg       0.92      0.92      0.92      1000
weighted avg       0.92      0.92      0.92      1000


Top 5 Features:
  Feature  1: 0.1561
  Feature  6: 0.1408
  Feature 10: 0.1248
  Feature  4: 0.0984
  Feature  5: 0.0673
```

### Hyperparameters ของ Gradient Boosting

การปรับแต่ง Gradient Boosting มีความละเอียดอ่อนกว่า Random Forest เนื่องจากมี Hyperparameters ที่สำคัญมากกว่า:

| Hyperparameter | คำอธิบาย | ค่าแนะนำเริ่มต้น | ผลกระทบ |
|---|---|---|---|
| `n_estimators` | จำนวนรอบ Boosting | 100-500 | มากเกินไป -> Overfit (ต่างจาก RF ที่ไม่ Overfit) |
| `learning_rate` | อัตราการเรียนรู้ | 0.01-0.3 | ค่าน้อยเรียนช้าแต่แม่นยำ ต้องเพิ่ม `n_estimators` |
| `max_depth` | ความลึกสูงสุดของ Tree | 3-8 | ค่าน้อยกว่า RF เพราะใช้ Weak Learners |
| `subsample` | สัดส่วนข้อมูลต่อรอบ | 0.7-1.0 | ช่วยลด Overfitting (คล้าย Bagging ของ RF) |
| `reg_alpha` / `reg_lambda` | Regularization (L1/L2) | 0-1 | ป้องกัน Overfitting โดยลงโทษ Tree ที่ซับซ้อนเกินไป |

**หมายเหตุสำคัญ:** สิ่งที่แตกต่างจาก Random Forest คือ Gradient Boosting **สามารถ Overfit ได้** หากเพิ่มจำนวนรอบ (`n_estimators`) มากเกินไป ดังนั้น ควรใช้ Early Stopping ร่วมกับ Validation Set เพื่อหยุดการฝึกสอนเมื่อ Performance บน Validation Set เริ่มลดลง

### เปรียบเทียบ Random Forest กับ Gradient Boosting

| คุณลักษณะ | Random Forest (Bagging) | Gradient Boosting |
|---|---|---|
| **สถาปัตยกรรม** | Parallel (ขนาน) | Sequential (ลำดับขั้น) |
| **เป้าหมาย** | ลด Variance | ลด Bias |
| **Tree แต่ละต้น** | Deep Trees (ลึก) | Shallow Trees (ตื้น, Weak Learners) |
| **ความเสี่ยง Overfit** | ต่ำ (เพิ่ม Trees ไม่ทำ Overfit) | สูง (ต้องใช้ Early Stopping) |
| **ความเร็วฝึกสอน** | เร็ว (Parallel) | ช้า (Sequential) |
| **การปรับ Hyperparameters** | ง่าย (ไม่ไวต่อ Tuning) | ยาก (ไวต่อ Learning Rate, Depth) |
| **Accuracy ทั่วไป** | ดี | ดีกว่าเล็กน้อย (หาก Tune ดี) |

**ตัวอย่างในบริบทประเทศไทย:**

- **ธนาคาร (Credit Scoring):** XGBoost เป็นที่นิยมมากในการพิจารณาสินเชื่อ เพราะให้ Accuracy สูงกว่า Random Forest 1-3% ซึ่งในปริมาณสินเชื่อหลายพันล้านบาทต่อปี ความแม่นยำที่เพิ่มขึ้นนี้มีมูลค่าทางธุรกิจสูงมาก
- **โรงพยาบาล (Disease Prediction):** LightGBM ถูกใช้ในระบบ Clinical Decision Support System (CDSS) ของโรงพยาบาลขนาดใหญ่ เนื่องจากสามารถประมวลผลข้อมูลผู้ป่วยจำนวนมากได้อย่างรวดเร็ว
- **E-commerce (Recommendation):** แพลตฟอร์ม E-commerce ชั้นนำใช้ Gradient Boosting ในระบบ Ranking ของสินค้า ซึ่งต้องประมวลผลแบบ Real-time

### แนวทางการเลือกระหว่าง RF และ GBM ในทางปฏิบัติ

```
เริ่มต้น
  |
  +-- ข้อมูลขนาดเล็ก (<10K) หรือต้องการ Baseline เร็ว?
  |      ---> ใช้ Random Forest (ปรับน้อย ผลดี)
  |
  +-- ต้องการ Accuracy สูงสุด? (มีเวลา Tune)
  |      ---> ใช้ XGBoost/LightGBM + Early Stopping + Grid Search
  |
  +-- ข้อมูลขนาดใหญ่ (>100K rows)?
  |      ---> ใช้ LightGBM (เร็วกว่า XGBoost)
  |
  +-- Feature มี Categorical มาก?
         ---> ใช้ CatBoost (ไม่ต้อง One-Hot Encoding)
```

### สรุปประเด็นสำคัญ (Key Takeaways)

- **Gradient Boosting** สร้างโมเดลแบบลำดับขั้น โดยแต่ละ Tree แก้ไข Residuals ของรอบก่อนหน้า ส่งผลให้ลด Bias ได้อย่างมีประสิทธิภาพ
- **XGBoost, LightGBM, CatBoost** เป็นไลบรารียอดนิยมที่ปรับปรุง Gradient Boosting ให้เร็วขึ้นและมี Regularization ป้องกัน Overfitting
- **Learning Rate** เป็น Hyperparameter ที่สำคัญที่สุด — ค่าน้อยให้ผลดีกว่าแต่ต้องเพิ่มจำนวนรอบ
- **Early Stopping** เป็นเทคนิคสำคัญที่ต้องใช้คู่กับ Gradient Boosting เพื่อป้องกัน Overfitting
- **การเลือกระหว่าง RF กับ GBM** ขึ้นอยู่กับบริบท: RF เหมาะเป็น Baseline ที่ปรับง่าย, GBM เหมาะเมื่อต้องการ Accuracy สูงสุดและมีเวลา Tune

### คำถามท้าย Section

1. Gradient Boosting มักใช้ Decision Tree ที่ตื้น (Shallow Trees, max_depth = 3-8) ในขณะที่ Random Forest มักใช้ Decision Tree ที่ลึก (Deep Trees) เหตุใดจึงเป็นเช่นนั้น? จงอธิบายความสัมพันธ์กับ Bias-Variance Tradeoff
2. หากนักศึกษาต้องสร้างโมเดลทำนายยอดขายรายเดือนของร้านค้าปลีกในจังหวัดขอนแก่น โดยมีข้อมูลธุรกรรมย้อนหลัง 500,000 รายการ และต้องการ Accuracy สูงสุด นักศึกษาจะเลือกใช้ Random Forest หรือ XGBoost/LightGBM? จงอธิบายเหตุผลและขั้นตอนการปรับ Hyperparameters ที่จะใช้
3. อธิบายว่าทำไม Gradient Boosting จึง Overfit ได้เมื่อเพิ่ม `n_estimators` มากเกินไป ในขณะที่ Random Forest ไม่มีปัญหานี้

## การตีความและนำเสนอผลลัพธ์ Ensemble Methods

เมื่อสร้างโมเดล Ensemble Methods ที่มี Accuracy สูงแล้ว สิ่งสำคัญไม่แพ้กันคือ **การตีความโมเดล** (Model Interpretation) เพื่อสื่อสารผลลัพธ์ให้ผู้มีอำนาจตัดสินใจ (Decision Makers) เข้าใจและนำไปใช้ประโยชน์ได้จริง โดยเฉพาะในบริบทธุรกิจที่ผู้บริหารอาจไม่มีความรู้ด้าน Machine Learning (Knaflic, 2015)

### Feature Importance: การตีความจาก RF และ XGBoost

ทั้ง Random Forest และ XGBoost มีกลไกวัดความสำคัญของ Feature (Feature Importance) แต่มีวิธีคำนวณที่แตกต่างกัน:

| วิธีการ | Random Forest | XGBoost |
|---|---|---|
| **Metric หลัก** | Mean Decrease in Impurity (MDI) | Gain / Weight / Cover |
| **หลักการ** | วัดว่า Feature ลด Gini Impurity ได้มากเฉลี่ยเท่าใดในทุก Trees | วัด Gain ที่ได้จากการ Split ด้วย Feature นั้น |
| **ข้อดี** | เข้าใจง่าย คำนวณเร็ว | มีหลาย Metric ให้เลือก |
| **ข้อจำกัด** | Bias ต่อ Feature ที่มีหลายค่า (Cardinal) | อาจให้ผลต่างจาก RF อย่างมาก |

**ตัวอย่างโค้ด: เปรียบเทียบ Feature Importance**

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# สร้างข้อมูลจำลอง: ทำนายลูกค้าที่จะยกเลิกใช้บริการ (Churn)
X, y = make_classification(n_samples=2000, n_features=8, n_informative=5,
                           random_state=42)
feature_names = ['อายุ', 'รายได้', 'ระยะเป็นลูกค้า', 'จำนวนร้องเรียน',
                 'ค่าบริการ/เดือน', 'จำนวนสัญญา', 'Feature_7', 'Feature_8']

# โมเดล 1: Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

# โมเดล 2: XGBoost
xgb = XGBClassifier(n_estimators=100, random_state=42, eval_metric='logloss')
xgb.fit(X, y)

# เปรียบเทียบ Feature Importance
print("Feature Importance Comparison:")
print(f"{'Feature':<20s} {'RF':>8s} {'XGBoost':>8s}")
print("-" * 40)
for name, rf_imp, xgb_imp in zip(feature_names, rf.feature_importances_,
                                  xgb.feature_importances_):
    print(f"{name:<20s} {rf_imp:>8.4f} {xgb_imp:>8.4f}")
```

**ผลลัพธ์:**

```
Feature Importance Comparison:
Feature                    RF  XGBoost
----------------------------------------
อายุ                   0.0827   0.0650
รายได้                 0.1106   0.1029
ระยะเป็นลูกค้า         0.0366   0.0340
จำนวนร้องเรียน         0.1942   0.2333
ค่าบริการ/เดือน        0.1482   0.1224
จำนวนสัญญา             0.1168   0.1155
Feature_7              0.0950   0.0734
Feature_8              0.2158   0.2535
```

> **หมายเหตุ:** RF และ XGBoost อาจจัดอันดับ Feature Importance **ต่างกัน** ดังนั้น ควรใช้เทคนิค Permutation Importance ด้วยเพื่อยืนยันผลลัพธ์

### จาก Feature Importance สู่ Actionable Insights

การได้ Feature Importance เพียงอย่างเดียวยังไม่เพียงพอ สิ่งที่ผู้บริหารต้องการคือ **Actionable Insights** — คำแนะนำที่สามารถนำไปปฏิบัติได้จริง

**กรอบ 3 ขั้นตอน: Data → Insight → Action**

```
[Feature Importance]      [Insight]                [Action]
====================      ====================     ========================
จำนวนร้องเรียน = 0.35  → ร้องเรียนมากยิ่งเลิก    → ปรับปรุง Call Center
ค่าบริการ/เดือน = 0.20  → ค่าบริการสูงเสี่ยง       → เสนอแพ็กเกจส่วนลด
ระยะเป็นลูกค้า = 0.18  → ลูกค้าใหม่เลิกง่าย      → โปรแกรม Onboarding
```

### Template การนำเสนอผลลัพธ์ต่อผู้บริหาร

เมื่อต้องนำเสนอผลลัพธ์ของ Ensemble Model ให้ผู้บริหารที่ไม่มีพื้นฐาน ML สิ่งสำคัญคือ **ไม่พูดเรื่อง Algorithm แต่พูดเรื่อง Business Impact**

**โครงสร้างการนำเสนอ:**

1. **ปัญหา (Problem):** "ปัจจุบันลูกค้ายกเลิกบริการ 15% ต่อไตรมาส คิดเป็นมูลค่า 50 ล้านบาท"
2. **สิ่งที่ค้นพบ (Findings):** "จากการวิเคราะห์ข้อมูลลูกค้า 50,000 ราย พบว่าปัจจัย 3 อันดับแรกที่ทำให้ลูกค้าเลิก คือ..."
3. **ความแม่นยำ (Confidence):** "โมเดลสามารถระบุลูกค้าที่จะเลิกได้ล่วงหน้า 30 วัน ด้วยความแม่นยำ 85%"
4. **คำแนะนำ (Recommendations):** "หากดำเนินการ 3 มาตรการ คาดว่าจะลดอัตราการเลิกลง 5% คิดเป็นมูลค่า ~17 ล้านบาท/ไตรมาส"

**สิ่งที่ไม่ควรพูดต่อผู้บริหาร:** - ✗ "โมเดล XGBoost ที่ max_depth=4 learning_rate=0.1 ให้ AUC-ROC = 0.92" - ✓ "ระบบ AI ของเราสามารถระบุลูกค้าที่เสี่ยงได้อย่างแม่นยำ 85% ก่อนที่พวกเขาจะตัดสินใจยกเลิก"

### กรณีศึกษา: Credit Scoring ในธนาคารไทย

พิจารณาสถานการณ์: ทีม Data Science ของธนาคารไทยแห่งหนึ่งสร้างโมเดล Random Forest เพื่อประเมินความเสี่ยงสินเชื่อ (Credit Risk)

**ขั้นตอนการนำเสนอ:**

1. **Feature Importance → Story:** "จากการวิเคราะห์ลูกค้า 100,000 ราย พบว่าปัจจัยที่มีอิทธิพลสูงสุดต่อการผิดนัดชำระ คือ อัตราส่วนหนี้สินต่อรายได้ (DTI Ratio) รองลงมาคือ ประวัติการชำระล่าช้า"
2. **Actionable Insight:** "ลูกค้าที่มี DTI > 0.5 มีโอกาสผิดนัดชำระสูงกว่า 4.2 เท่า → ควรกำหนดเกณฑ์ DTI ไม่เกิน 0.5 สำหรับสินเชื่อส่วนบุคคล"
3. **Business Impact:** "หากนำโมเดลไปใช้จริง สามารถลดหนี้เสีย (NPL) ลงได้ประมาณ 120 ล้านบาท/ปี"

### สรุปประเด็นสำคัญ

- **Feature Importance** เป็นเครื่องมือหลักในการตีความ Ensemble Methods แต่โมเดลต่างกันอาจให้ผลต่างกัน ควรใช้หลายวิธียืนยัน
- **Actionable Insights** สำคัญกว่า Technical Metrics — ผู้บริหารต้องการคำแนะนำที่ปฏิบัติได้ ไม่ใช่ค่า AUC-ROC
- **Template: Problem → Findings → Confidence → Recommendations** ช่วยจัดโครงสร้างการนำเสนอให้กระชับ
- **พูดภาษาธุรกิจ ไม่พูดภาษา Algorithm** — แปลงทุกผลลัพธ์เป็นมูลค่าเงินหรือ Impact ทางธุรกิจ

### คำถามท้าย Section

1. หากโมเดล Random Forest ให้ Feature Importance สูงสุดคือ "จำนวนธุรกรรม" ในการทำนาย Churn ของลูกค้า แต่ผู้บริหารพบว่าจำนวนธุรกรรมสูงเป็นเพราะมีการเชื่อมสัญญาหลายรายการก่อนยกเลิก ปัญหานี้เรียกว่าอะไร? (Data Leakage) นักศึกษาจะตรวจจับได้อย่างไร?
2. จงเขียนสรุปการนำเสนอผลลัพธ์ Ensemble Model สำหรับสถานการณ์: โรงพยาบาลต้องการทำนายผู้ป่วยที่มีความเสี่ยงจะ Re-admit ภายใน 30 วัน โดยใช้โครงสร้าง Problem → Findings → Confidence → Recommendations

### สรุปท้ายบท

ในบทนี้ นักศึกษาได้เรียนรู้เกี่ยวกับ: - **Ensemble Methods:** รวมโมเดลหลายตัวเพื่อลดข้อจำกัดของโมเดลเดี่ยว — 3 ประเภท: Bagging, Boosting, Stacking - **Random Forest:** ใช้ Bagging + Random Feature Selection สร้าง Trees อิสระ → ลด Variance - **Gradient Boosting:** สร้าง Trees แบบลำดับขั้น แก้ Residuals → ลด Bias; XGBoost/LightGBM เป็นไลบรารียอดนิยม - **การเลือก RF vs GBM:** RF เหมาะเป็น Baseline ปรับง่าย; GBM เหมาะเมื่อต้องการ Accuracy สูงสุด - **การตีความ:** สื่อสารผลลัพธ์ด้วย Feature Importance → Actionable Insights → Business Value

**คำศัพท์ที่สำคัญ:** ภูมิปัญญาของฝูงชน (Wisdom of the Crowds), ปัญหาของ Single Model และข้อจำกัดของ Dec, สถาปัตยกรรมหลักของ Ensemble Methods, Bootstrap Aggregating (Bagging), การสุ่มเลือกคุณลักษณะ (Random Subsets of, ความสำคัญของคุณลักษณะ (Feature Importanc, พารามิเตอร์ควบคุมการเรียนรู้ (Hyperparam, พฤติกรรมการเรียนรู้และปัญหา Overfitting

### เอกสารอ้างอิง

- Grus, J. (2019). *Data Science from Scratch: First Principles with Python* (2nd ed.). O'Reilly Media.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python* (2nd ed.). Springer.
- Knaflic, C. N. (2015). *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

### ✏ แบบฝึกหัดท้ายบท

#### ส่วนที่ 1: คำถามทบทวน

*คำชี้แจง: จงเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว*

**1 แนวคิดหลักของ Ensemble Methods คืออะไร**

- A การใช้โมเดลที่มีความซับซ้อนสูงเพียงโมเดลเดียวเพื่อลด Bias
- B การสร้างโมเดลที่เรียนรู้จากความผิดพลาดของโมเดลก่อนหน้าเสมอ
- C การรวมผลลัพธ์จากหลายโมเดล (Multiple Weak Learners) เพื่อสร้างโมเดลที่มีประสิทธิภาพสูงขึ้น
- D การลดจำนวน Feature ในชุดข้อมูลเพื่อลดระยะเวลาในการประมวลผล

▸ แสดงเฉลย

**2 เทคนิคใดที่ Random Forest ใช้ในการสุ่มข้อมูลเพื่อสร้าง Decision Tree แต่ละต้น**

- A Cross-Validation
- B Bootstrapping
- C Feature Selection
- D Stacking

▸ แสดงเฉลย

**3 เหตุใด Random Forest จึงต้องทำการสุ่ม Feature (Feature Subsampling) ในแต่ละ Node ของการสร้าง Tree**

- A เพื่อลดการใช้หน่วยความจำของระบบ
- B เพื่อทำให้ Tree แต่ละต้นมีความแตกต่างกัน (Decorrelate Trees) และลด Variance
- C เพื่อบังคับให้ Tree มีความลึก (Depth) ลดลง
- D เพื่อเพิ่มความเร็วในการทำ Prediction

▸ แสดงเฉลย

**4 ข้อใดคือข้อได้เปรียบที่สำคัญที่สุดของ Single Decision Tree เมื่อเปรียบเทียบกับ Random Forest**

- A มีความแม่นยำ (Accuracy) สูงกว่าในชุดข้อมูลที่มีความซับซ้อน
- B ทนทานต่อ Overfitting ได้ดีกว่า
- C สามารถอธิบายเหตุผลในการตัดสินใจ (Interpretability) ได้ง่ายและชัดเจนกว่า
- D รองรับข้อมูลที่มี Missing Values ได้โดยไม่ต้องทำ Imputation

▸ แสดงเฉลย

**5 หากธนาคารแห่งหนึ่งในประเทศไทยต้องการสร้างโมเดลพิจารณาอนุมัติสินเชื่อ โดยมีข้อกำหนดทางกฎหมายว่าจะต้องสามารถอธิบายเหตุผลของการปฏิเสธสินเชื่อให้ลูกค้าทราบได้เป็นรายบุคคล (Rule-based explanation) โมเดลใดเหมาะสมที่สุด**

- A Random Forest ที่มี 500 Trees
- B Single Decision Tree ที่มีการทำ Pruning อย่างเหมาะสม
- C Gradient Boosting Machines
- D Ensemble Voting Classifier

▸ แสดงเฉลย

#### ส่วนที่ 2: คำถามอัตนัย

*คำชี้แจง: จงอธิบาย วิเคราะห์ หรือเปรียบเทียบตามประเด็นที่กำหนด*

**1 การประนีประนอมระหว่าง Bias และ Variance (Bias-Variance Tradeoff):**

จงอธิบายกลไกทางคณิตศาสตร์หรือสถิติที่อธิบายว่า เหตุใด Random Forest จึงสามารถลด Variance ของโมเดลได้ โดยไม่ทำให้ Bias เพิ่มขึ้นอย่างมีนัยสำคัญ เมื่อเทียบกับการใช้ Decision Tree เพียงต้นเดียว

▸ แสดงแนวคำตอบ

**2 การวิเคราะห์บริบทการใช้งาน (Contextual Analysis):**

โรงพยาบาลศรีนครินทร์ มหาวิทยาลัยขอนแก่น ต้องการสร้างโมเดลพยากรณ์ความเสี่ยงโรคเบาหวานของผู้ป่วย หากนักศึกษาเป็น Data Scientist ของโครงการนี้ จงเปรียบเทียบข้อดีและข้อเสียของการใช้ Single Decision Tree กับ Random Forest ในบริบททางการแพทย์ และนักศึกษาจะเลือกใช้โมเดลใด พร้อมให้เหตุผลประกอบ

▸ แสดงแนวคำตอบ

**3 ผลกระทบของ Hyperparameters:**

ในโมเดล Random Forest พารามิเตอร์ `n_estimators` (จำนวนต้นไม้) และ `max_depth` (ความลึกสูงสุดของต้นไม้) มีผลต่อประสิทธิภาพของโมเดลอย่างไร หากกำหนดค่าทั้งสองสูงเกินไปหรือต่ำเกินไป จะเกิดผลลัพธ์อย่างไร

▸ แสดงแนวคำตอบ

#### ส่วนที่ 3: แบบฝึกหัด Coding

*คำชี้แจง: ให้เขียนโปรแกรมภาษา Python เพื่อแก้ปัญหาตามโจทย์ที่กำหนด*

##### ข้อที่ 1: การเปรียบเทียบ Single Tree และ Random Forest

**โจทย์:** สมมติว่านักศึกษาได้รับชุดข้อมูลจำลอง (Synthetic Data) เกี่ยวกับการพิจารณาให้ทุนการศึกษาของนักศึกษามหาวิทยาลัยขอนแก่น โดยมี Feature เช่น เกรดเฉลี่ย (GPA), ชั่วโมงกิจกรรม, รายได้ครอบครัว ฯลฯ จงเขียนโค้ดเพื่อสร้างข้อมูลจำลองจำนวน 1,000 รายการ จากนั้นสร้างและประเมินผลโมเดล `DecisionTreeClassifier` และ `RandomForestClassifier` เพื่อเปรียบเทียบค่าความแม่นยำ (Accuracy) บน Test Set

```python
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. สร้างชุดข้อมูลจำลอง (นักศึกษา 1000 คน, 5 Features)
X, y = make_classification(n_samples=1000, n_features=5, n_informative=3,
                           n_redundant=1, random_state=42,
                           weights=[0.7, 0.3]) # 70% ไม่ได้ทุน, 30% ได้ทุน

# ตั้งชื่อ Feature เพื่อให้เห็นภาพชัดเจน
feature_names = ['GPA', 'Activity_Hours', 'Family_Income', 'Distance_to_Uni', 'Sibling_Count']
df_X = pd.DataFrame(X, columns=feature_names)

# 2. แบ่งข้อมูล Train (80%) และ Test (20%)
X_train, X_test, y_train, y_test = train_test_split(df_X, y, test_size=0.2, random_state=42)

# 3. สร้างและฝึกสอนโมเดล Single Decision Tree
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X_train, y_train)
tree_preds = tree_model.predict(X_test)
tree_acc = accuracy_score(y_test, tree_preds)

# 4. สร้างและฝึกสอนโมเดล Random Forest (ใช้ 100 ต้น)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_preds)

# 5. แสดงผลเปรียบเทียบ
print(f"ความแม่นยำของ Single Decision Tree: {tree_acc:.4f}")
print(f"ความแม่นยำของ Random Forest: {rf_acc:.4f}")
```

**ผลลัพธ์:**

```
ความแม่นยำของ Single Decision Tree: 0.9300
ความแม่นยำของ Random Forest: 0.9700
```

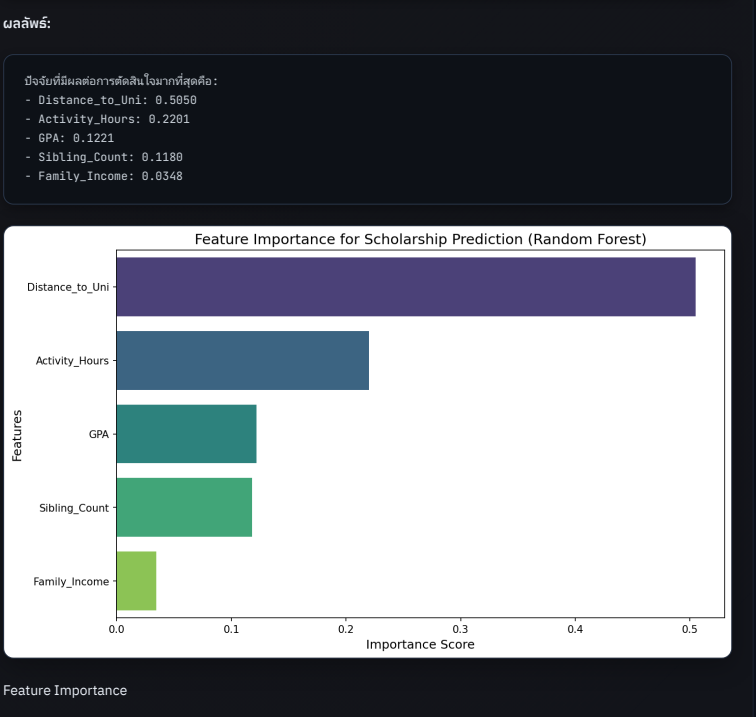
##### ข้อที่ 2: การสกัด Feature Importance จาก Random Forest

**โจทย์:** จากบริษัท E-commerce แห่งหนึ่งในประเทศไทย ต้องการทราบว่าปัจจัยใดมีผลต่อการยกเลิกการใช้งานแอปพลิเคชันของลูกค้า (Customer Churn) จงใช้โมเดล Random Forest จากข้อ 1 มาทำการดึงค่าความสำคัญของตัวแปร (Feature Importance) และสร้างกราฟแท่ง (Bar Chart) เพื่อแสดงผลลัพธ์เรียงจากปัจจัยที่สำคัญมากที่สุดไปยังน้อยที่สุด

```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. ดึงค่า Feature Importance จากโมเดล Random Forest ที่ train ไว้แล้ว
importances = rf_model.feature_importances_

# 2. สร้าง DataFrame เพื่อให้ง่ายต่อการจัดการและจัดเรียงข้อมูล
feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
})

# 3. จัดเรียงข้อมูลจากค่าความสำคัญมากไปน้อย
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# 4. สร้างกราฟแท่ง (Bar Chart)
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, palette='viridis')

# 5. ตกแต่งกราฟให้สวยงามและอ่านง่าย
plt.title('Feature Importance for Scholarship Prediction (Random Forest)', fontsize=14)
plt.xlabel('Importance Score', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.tight_layout()

# แสดงผลกราฟ
plt.show()

# พิมพ์อธิบายผลลัพธ์
print("ปัจจัยที่มีผลต่อการตัดสินใจมากที่สุดคือ:")
for index, row in feature_importance_df.iterrows():
    print(f"- {row['Feature']}: {row['Importance']:.4f}")
```

**ผลลัพธ์:**

```
ปัจจัยที่มีผลต่อการตัดสินใจมากที่สุดคือ:
- Distance_to_Uni: 0.5050
- Activity_Hours: 0.2201
- GPA: 0.1221
- Sibling_Count: 0.1180
- Family_Income: 0.0348
```



Feature Importance

← บทที่ 9: Classification — การจำแนกประเภท
บทที่ 11: Clustering — การจัดกลุ่มข้อมูล →